```
from google.colab import files
uploaded = files.upload()  # select data.zip

import zipfile
with zipfile.ZipFile("data.zip", "r") as z:
    z.extractall("data")
print("✅ Unzipped")
```

```
Choose files data.zip
data.zip(application/zip) - 5105676 bytes, last modified: 18/04/2026 - 100% done
Saving data.zip to data (1).zip
✅ Unzipped
```

```
import os
import numpy as np

DATA_PATH = "data/data"
actions = sorted(os.listdir(DATA_PATH))
actions = [a for a in actions if os.path.isdir(os.path.join(DATA_PATH

print("=" * 50)
print(f"  Total actions found: {len(actions)}")
print("=" * 50)

total_sequences = 0
for action in actions:
    seqs = os.listdir(os.path.join(DATA_PATH, action))
    seqs = [s for s in seqs if os.path.isdir(os.path.join(DATA_PATH,
    count = len(seqs)
    total_sequences += count
    bar = "█" * count
    print(f"  {action:<10}  {count:>3} sequences   {bar}")

print("=" * 50)
print(f"  TOTAL: {total_sequences} sequences across {len(actions)} ac
```

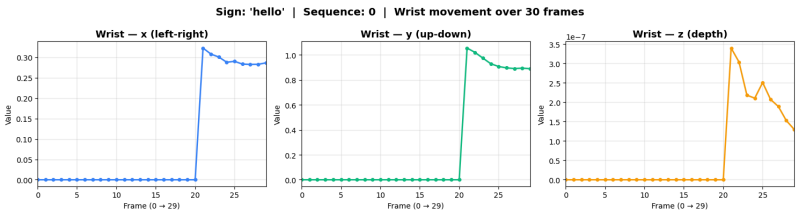
```
==================================================
  Total actions found: 10
==================================================
  call         20 sequences   ████████████████████
  fist         20 sequences   ████████████████████
  hello        20 sequences   ████████████████████
  no           20 sequences   ████████████████████
  ok           20 sequences   ████████████████████
  peace        20 sequences   ████████████████████
  point        20 sequences   ████████████████████
  stop         20 sequences   ████████████████████
  thanks       20 sequences   ████████████████████
  yes          20 sequences   ████████████████████
==================================================
  TOTAL: 200 sequences across 10 actions
```

```
# Pick first action, first sequence, first frame
action = actions[0]
seq_path = os.path.join(DATA_PATH, action)
first_seq = sorted(os.listdir(seq_path))[0]
frame_path = os.path.join(seq_path, first_seq, "0.npy")

frame = np.load(frame_path)

print(f"Action   : {action}")
print(f"Sequence : {first_seq}")
print(f"Frame    : 0.npy")
print(f"Shape    : {frame.shape}  ← should be (63,)")
print(f"= 21 landmarks × 3 (x, y, z) = 63 values")
print()
print("First 9 values (landmark 0 = wrist):")
print(f"  x = {frame[0]:.4f}  (left-right position)")
print(f"  y = {frame[1]:.4f}  (up-down position)")
print(f"  z = {frame[2]:.4f}  (depth toward camera)")
print()
print("Full 63 values:")
print(frame.reshape(21, 3))
```

```
Action   : call
Sequence : 0
Frame    : 0.npy
Shape    : (63,)  ← should be (63,)
= 21 landmarks × 3 (x, y, z) = 63 values

First 9 values (landmark 0 = wrist):
  x = 0.0000  (left-right position)
  y = 0.0000  (up-down position)
  z = 0.0000  (depth toward camera)

Full 63 values:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

```
import matplotlib.pyplot as plt
```

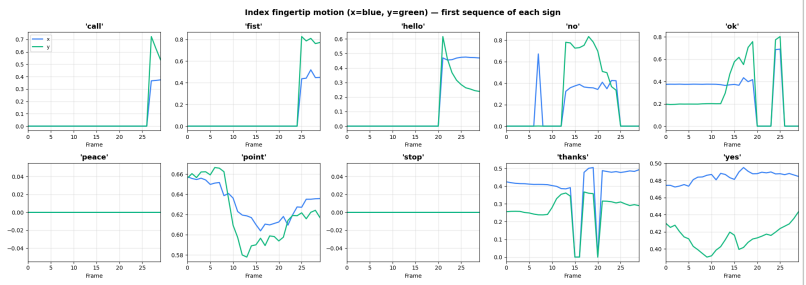
```
action = "hello"   # change this to any of your signs
seq_folder = sorted(os.listdir(os.path.join(DATA_PATH, action)))[0]

frames = []
for i in range(30):
    f = np.load(os.path.join(DATA_PATH, action, seq_folder, f"{i}.npy
    frames.append(f)

frames = np.array(frames)  # shape (30, 63)

# Plot x, y, z of wrist (landmark 0) across 30 frames
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
labels = ["x (left-right)", "y (up-down)", "z (depth)"]
colors = ["#3B82F6", "#10B981", "#F59E0B"]

for i, (label, color) in enumerate(zip(labels, colors)):
    axes[i].plot(frames[:, i], color=color, linewidth=2, marker="o",
    axes[i].set_title(f"Wrist — {label}", fontsize=13, fontweight="bo
    axes[i].set_xlabel("Frame (0 → 29)")
    axes[i].set_ylabel("Value")
    axes[i].grid(True, alpha=0.3)
    axes[i].set_xlim(0, 29)

plt.suptitle(f"Sign: '{action}'  |  Sequence: {seq_folder}  |  Wrist
             fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()
print("This is the motion of the wrist across 1 second of gesture")
```




```
This is the motion of the wrist across 1 second of gesture
```

```
fig, axes = plt.subplots(2, 5, figsize=(20, 7))
axes = axes.flatten()

for idx, action in enumerate(sorted(actions)):
    seq_folder = sorted(os.listdir(os.path.join(DATA_PATH, action)))[
    frames = []
    for i in range(30):
        f = np.load(os.path.join(DATA_PATH, action, seq_folder, f"{i}
        frames.append(f)
    frames = np.array(frames)

    # Plot index fingertip (landmark 8) x and y
    axes[idx].plot(frames[:, 24], color="#3B82F6", linewidth=2, label
    axes[idx].plot(frames[:, 25], color="#10B981", linewidth=2, label
    axes[idx].set_title(f"'{action}'", fontsize=13, fontweight="bold"
```

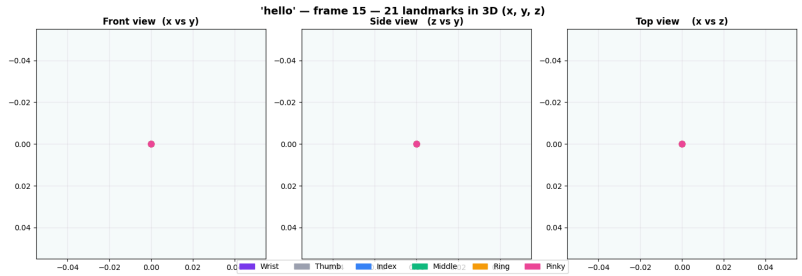
```
    axes[idx].set_xlabel("Frame")
    axes[idx].grid(True, alpha=0.3)
    axes[idx].set_xlim(0, 29)
    if idx == 0:
        axes[idx].legend(fontsize=9)

plt.suptitle("Index fingertip motion (x=blue, y=green) — first sequen
             fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()
print("Each sign has a unique motion pattern — this is what LSTM lear
```




```
Each sign has a unique motion pattern — this is what LSTM learns to di
```

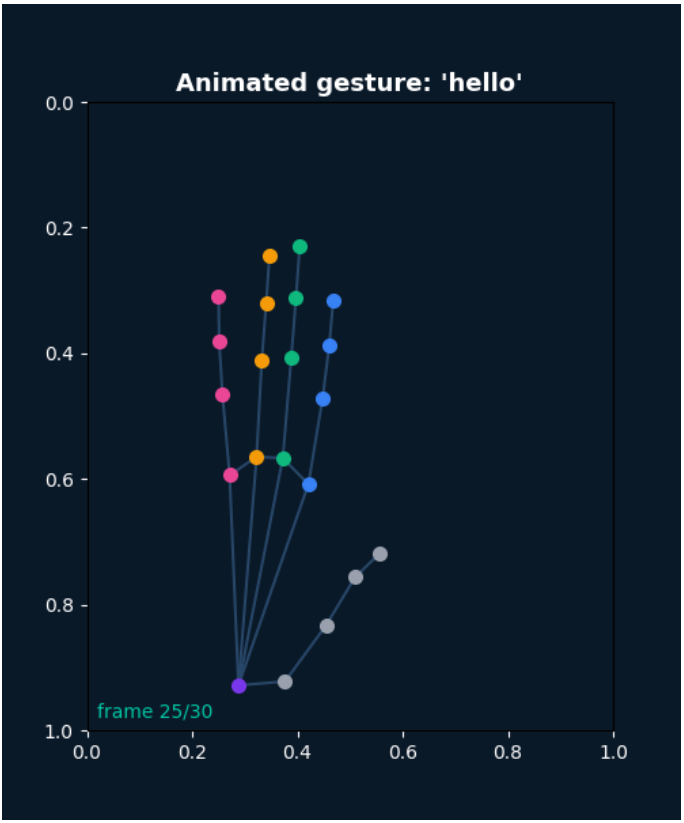
```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

HAND_CONNECTIONS = [
    (0,1),(1,2),(2,3),(3,4),
    (0,5),(5,6),(6,7),(7,8),
    (0,9),(9,10),(10,11),(11,12),
    (0,13),(13,14),(14,15),(15,16),
    (0,17),(17,18),(18,19),(19,20),
    (5,9),(9,13),(13,17)
]
FINGER_COLORS = {
    "thumb":  ([0,1,2,3,4],       "#9CA3AF"),
    "index":  ([0,5,6,7,8],       "#3B82F6"),
    "middle": ([0,9,10,11,12],    "#10B981"),
    "ring":   ([0,13,14,15,16],   "#F59E0B"),
    "pinky":  ([0,17,18,19,20],   "#EC4899"),
}

action = "hello"
seq_folder = sorted(os.listdir(os.path.join(DATA_PATH, action)))[0]
frame_idx = 15  # middle of gesture

frame = np.load(os.path.join(DATA_PATH, action, seq_folder, f"{frame_
pts = frame.reshape(21, 3)  # (21, 3)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

```
views = [
    ("Front view  (x vs y)", 0, 1),
    ("Side view   (z vs y)", 2, 1),
    ("Top view    (x vs z)", 0, 2),
]

for ax, (title, xi, yi) in zip(axes, views):
    for (a, b) in HAND_CONNECTIONS:
        ax.plot([pts[a, xi], pts[b, xi]], [pts[a, yi], pts[b, yi]],
                color="#CBD5E1", linewidth=1.5, zorder=1)

    finger_colors = {
        0: "#7C3AED",
        **{j: "#9CA3AF" for j in range(1, 5)},
        **{j: "#3B82F6" for j in range(5, 9)},
        **{j: "#10B981" for j in range(9, 13)},
        **{j: "#F59E0B" for j in range(13, 17)},
        **{j: "#EC4899" for j in range(17, 21)},
    }
    for i, pt in enumerate(pts):
        ax.scatter(pt[xi], pt[yi], color=finger_colors[i], s=60, zord

    ax.set_title(title, fontsize=12, fontweight="bold")
    ax.set_aspect("equal")
    ax.invert_yaxis()
    ax.grid(True, alpha=0.2)
    ax.set_facecolor("#F8FAFC")

patches = [
    mpatches.Patch(color="#7C3AED", label="Wrist"),
    mpatches.Patch(color="#9CA3AF", label="Thumb"),
    mpatches.Patch(color="#3B82F6", label="Index"),
    mpatches.Patch(color="#10B981", label="Middle"),
    mpatches.Patch(color="#F59E0B", label="Ring"),
    mpatches.Patch(color="#EC4899", label="Pinky"),
]
fig.legend(handles=patches, loc="lower center", ncol=6, fontsize=10)
plt.suptitle(f"'{action}' — frame {frame_idx} — 21 landmarks in 3D (x
             fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()
```




```
from matplotlib.animation import FuncAnimation
```

```
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

action = "hello"   # change to any sign
seq_folder = sorted(os.listdir(os.path.join(DATA_PATH, action)))[0]

all_frames = []
for i in range(30):
    f = np.load(os.path.join(DATA_PATH, action, seq_folder, f"{i}.npy"
    all_frames.append(f.reshape(21, 3))

fig, ax = plt.subplots(figsize=(5, 6))
ax.set_xlim(0, 1); ax.set_ylim(0, 1)
ax.invert_yaxis()
ax.set_facecolor("#0D1B2A")
ax.set_title(f"Animated gesture: '{action}'", color="white", fontsize=
ax.tick_params(colors="white"); fig.patch.set_facecolor("#0D1B2A")

dot_colors = {0:"#7C3AED", **{j:"#9CA3AF" for j in range(1,5)},
              **{j:"#3B82F6" for j in range(5,9)}, **{j:"#10B981" for
              **{j:"#F59E0B" for j in range(13,17)}, **{j:"#EC4899" fo

lines = [ax.plot([], [], color="#2A4A6A", linewidth=1.5)[0] for _ in HA
dots  = [ax.plot([], [], "o", color=dot_colors[i], markersize=7)[0] fo
frame_label = ax.text(0.02, 0.02, "", color="#00C9A7", fontsize=10, tra

def update(f):
    pts = all_frames[f]
    for line, (a, b) in zip(lines, HAND_CONNECTIONS):
        line.set_data([pts[a,0], pts[b,0]], [pts[a,1], pts[b,1]])
    for dot, pt in zip(dots, pts):
        dot.set_data([pt[0]], [pt[1]])
    frame_label.set_text(f"frame {f+1}/30")
    return lines + dots + [frame_label]

ani = FuncAnimation(fig, update, frames=30, interval=80, blit=True)
plt.close()
HTML(ani.to_jshtml())
```




```
print("=" * 60)
print("  DATASET SUMMARY — Sign Language Project")
print("=" * 60)

total = 0
for action in sorted(actions):
    seq_path = os.path.join(DATA_PATH, action)
    seqs = [s for s in os.listdir(seq_path) if os.path.isdir(os.path.j
    n = len(seqs)
    total += n

    # Check frame count of first sequence
    first_frames = sorted(os.listdir(os.path.join(seq_path, seqs[0])))
    n_frames = len(first_frames)

    # Load one frame to get feature shape
    sample = np.load(os.path.join(seq_path, seqs[0], "0.npy"))
```

```
        print(f"  {action:<10}  {n:>3} sequences  × {n_frames} frames  × {s

print("−" * 60)
print(f"  Total sequences : {total}")
print(f"  Input shape     : (30, 63)")
print(f"  How to read     : 30 frames × 21 landmarks × 3 (x,y,z)")
print(f"  Storage format  : numpy .npy files (one per frame)")
print("=" * 60)
```

```
============================================================
  DATASET SUMMARY — Sign Language Project
============================================================
  call         20 sequences  × 30 frames  × 63 features
  fist         20 sequences  × 30 frames  × 63 features
  hello        20 sequences  × 30 frames  × 63 features
  no           20 sequences  × 30 frames  × 63 features
  ok           20 sequences  × 30 frames  × 63 features
  peace        20 sequences  × 30 frames  × 63 features
  point        20 sequences  × 30 frames  × 63 features
  stop         20 sequences  × 30 frames  × 63 features
  thanks       20 sequences  × 30 frames  × 63 features
  yes          20 sequences  × 30 frames  × 63 features
------------------------------------------------------------
  Total sequences : 200
  Input shape     : (30, 63)
  How to read     : 30 frames × 21 landmarks × 3 (x,y,z)
  Storage format  : numpy .npy files (one per frame)
============================================================
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.